

Propertypeak – Git Basics & Change Tracking (Printable)

This document explains **how Git helps you track, understand, and control code changes.**

PURPOSE OF GIT (IN SIMPLE WORDS)

Git answers these questions: - What code changed? - When did it change? - Why did it change? - Can I undo it safely?

Git is your **memory and safety net** for code.

WHAT GIT TRACKS

Git tracks: - PHP files - Blade templates - JavaScript - CSS - Config files

Git does NOT track: - Database data - Images & uploads - CMS admin changes

(This is intentional.)

THE ONLY GIT COMMANDS YOU NEED

1 Check current changes

```
git status
```

Tells you: - Which files changed - Whether changes are saved (committed) or not

2 See exact differences

```
git diff
```

Shows: - What lines were added - What lines were removed

Always review before committing.

3 Save a checkpoint (commit)

```
git commit -m "Clear description of change"
```

Think of this as **Save Game**.

Good commit messages explain: - What changed - Why it changed

4 View history

```
git log
```

Shows: - All past changes - Who made them - When they happened

5 Undo a bad change (safe way)

```
git revert <commit-id>
```

This: - Reverses a change - Does not break history - Is safe for production

HOW GIT FITS YOUR WORKFLOW

1. Sync staging data
 2. Change code
 3. Review changes (git diff)
 4. Commit changes
 5. Test on staging
 6. Deploy same commit to production
-

GOLDEN GIT RULES

- Never commit untested code
 - Never commit directly on production as an experiment
 - Small, clear commits are safer than big ones
-

MENTAL MODEL

- Git = code timeline
- Commits = checkpoints
- Revert = undo button

FINAL RULE

If you can't explain a commit, don't deploy it.